

Estructuras de Datos Persistentes

Benjamín Letelier

Agenda

1. **Repaso:** Segment Tree normal
2. Segment Tree Persistente con *Path Copying*
3. Tipos de persistencia
4. Persistencia y análisis amortizado
5. **Problema G:** Persistent Queue

Segment Tree — ¿Qué es?

Estructura para responder **consultas por rango** y **actualizaciones puntuales** en tiempo logarítmico.

- Range sum query: `sum(l, r)`
- Range minimum query: `min(l, r)`
- Point update: `a[pos] = x`

Cada operación: **$O(\log n)$**

Ideal cuando tenemos muchas consultas y actualizaciones entremezcladas.

Segment Tree — Estructura

Árbol binario completo donde cada hoja es un elemento del array.

- Raíz (nodo 1): cubre todo el rango $[0, n-1]$
- Hijo izquierdo $(2*i)$: mitad izquierda
- Hijo derecho $(2*i+1)$: mitad derecha
- Cada nodo guarda un valor agregado (suma, mínimo, etc.)

Memoria: $4 * n$ posiciones en un array.

Segment Tree — Build $O(n)$

Construcción recursiva *bottom-up* (o *top-down* con combinación al volver):

```
int build(int i, int l, int r) {  
    if (l == r) return seg[i] = a[l];  
    int m = (l + r) / 2;  
    return seg[i] = build(2*i, l, m)  
                    + build(2*i+1, m+1, r);  
}
```

Cada elemento sube por el árbol exactamente una vez $\rightarrow O(n)$.

Segment Tree — Update $O(\log n)$

Bajamos a la hoja correspondiente y actualizamos los nodos en el camino de regreso:

```
void update(int i, int l, int r, int pos, int val) {  
    if (l == r) { seg[i] = val; return; }  
    int m = (l + r) / 2;  
    if (pos <= m) update(2*i, l, m, pos, val);  
    else update(2*i+1, m+1, r, pos, val);  
    seg[i] = seg[2*i] + seg[2*i+1];  
}
```

Solo tocamos $O(\log n)$ nodos por actualización.

Segment Tree — Query $O(\log n)$

Visitamos solo los nodos cuyos rangos están completamente cubiertos por `[ql, qr]` :

```
int query(int i, int l, int r, int ql, int qr) {  
    if (qr < l || r < ql) return 0;           // sin solapamiento  
    if (ql <= l && r <= qr) return seg[i];    // completamente dentro  
    int m = (l + r) / 2;  
    return query(2*i, l, m, ql, qr)  
        + query(2*i+1, m+1, r, ql, qr);  
}
```

El problema

Cada `update` **sobrescribe** el valor anterior.

No hay forma de consultar **versiones anteriores** del arreglo.

Si necesitamos responder consultas como:

| "¿Cuál era la suma en `[l, r]` después de las primeras `k` operaciones?"

...necesitamos **persistencia**.

Persistencia — Idea intuitiva

Queremos un *time machine* para nuestra estructura de datos:

- Cada update genera una **nueva versión**
- Podemos consultar **cualquier versión anterior**
- Las versiones comparten todo lo que no cambió

Observación clave: En un segment tree, cada update solo modifica $O(\log n)$ nodos. El resto del árbol permanece igual.

Segment Tree Persistente — Path Copying

En vez de sobrescribir nodos, **creamos nuevos nodos** en el camino de la raíz a la hoja modificada.

- Los nodos **fuera del camino** se **comparten** con la versión anterior
- Solo se crean **$O(\log n)$** nodos nuevos por update
- La versión anterior queda intacta

¿Por qué no sirve el array implícito?

En un segment tree normal usamos $2*i$ y $2*i+1$ para los hijos.

Con persistencia, **varias versiones comparten subárboles**.

Un mismo nodo puede ser hijo derecho en una versión e hijo izquierdo en otra.

Solución: cada nodo guarda explícitamente los **índices** de sus hijos (`left` , `right`) en un *node pool*.

Implementación — Node Pool

Usamos un arreglo global de nodos y un contador:

```
struct Node {  
    int val;    // valor agregado (suma, min, etc.)  
    int L, R;   // índices de hijos en el pool  
    Node() : val(0), L(0), R(0) {}  
};  
  
Node pool[MAX_NODES];  
int node_cnt = 0;  
  
int new_node(int v = 0, int l = 0, int r = 0) {  
    pool[++node_cnt] = {v, l, r};  
    return node_cnt;  
}
```

Implementación — Array de raíces

Cada versión se identifica por el índice de su **nodo raíz**:

```
int roots[MAX_VERSIONS];    // roots[v] = índice del nodo raíz de la versión v
int version_count = 0;
```

- Versión 0: estado inicial (construida con `build`)
- Versión 1: después del primer update
- Versión `v`: después del `v`-ésimo update sobre alguna versión base

Build — Versión 0

Igual que el segment tree normal, pero con node pool:

```
int build(int l, int r) {  
    if (l == r) return new_node(a[l]);  
    int m = (l + r) / 2;  
    int left = build(l, m);  
    int right = build(m + 1, r);  
    return new_node(pool[left].val + pool[right].val, left, right);  
}
```

$2n - 1$ nodos para la versión inicial.

Update — Creando nueva versión

Recibimos la raíz de la versión base y devolvemos la nueva raíz:

```
int update(int prev, int l, int r, int pos, int val) {
    if (l == r) return new_node(val);
    int m = (l + r) / 2;
    int L = pool[prev].L, R = pool[prev].R;
    if (pos <= m)
        L = update(L, l, m, pos, val);        // nuevo hijo izquierdo
    else
        R = update(R, m + 1, r, pos, val);    // nuevo hijo derecho
    return new_node(pool[L].val + pool[R].val, L, R);
}
```

Se crean $O(\log n)$ nodos. El hijo no modificado se comparte.

Query — Idéntico al normal

Solo necesitamos la raíz de la versión a consultar:

```
int query(int node, int l, int r, int ql, int qr) {  
    if (qr < l || r < ql) return 0;  
    if (ql <= l && r <= qr) return pool[node].val;  
    int m = (l + r) / 2;  
    return query(pool[node].L, l, m, ql, qr)  
        + query(pool[node].R, m + 1, r, ql, qr);  
}
```

Para consultar la versión `v`: `query(roots[v], 0, n-1, ql, qr)`.

Complejidad

	Tiempo	Nodos nuevos
Build	$O(n)$	$2n - 1$
Update	$O(\log n)$	$\log_2(n) + 1$
Query	$O(\log n)$	0

Memoria total: $O(n + Q \log n)$, donde Q = cantidad de updates.

Para $n = 10^5$ y $Q = 10^5$: $\rightarrow \sim 2 \times 10^6$ nodos.

Asignar entre $n * 20$ y $n * 40$ suele ser seguro.

Aplicación clásica

K-ésimo menor en un subarray `[l, r]` (sin updates):

- Segment tree sobre **valores** (coordenados), no sobre índices
- Versión `i` = insertar `a[i]` en versión `i-1`
- Para consultar `[l, r]` : restar versión `l-1` de versión `r`

```
int kth(int vl, int vr, int l, int r, int k) {  
    if (l == r) return l;  
    int m = (l + r) / 2;  
    int cntL = pool[pool[vr].L].val - pool[pool[vl].L].val;  
    if (k <= cntL)  
        return kth(pool[vl].L, pool[vr].L, l, m, k);  
    else  
        return kth(pool[vl].R, pool[vr].R, m + 1, r, k - cntL);  
}
```

Tipos de Persistencia

No toda persistencia es igual. Hay cuatro niveles:

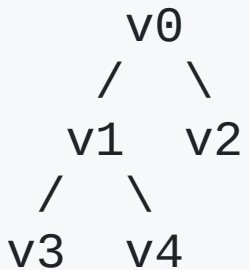
Tipo	Modificar versiones	Merge	Timeline
Parcial	Solo la última	No	Línea
Completa	Cualquiera	No	Árbol
Confluente	Cualquiera	Sí	DAG
Funcional	Ninguna (immutable)	No	Árbol

Persistencia Parcial

- Solo se puede modificar la **versión más reciente**
- Se pueden consultar todas las versiones
- El timeline es una **línea recta**: `v0 → v1 → v2 → . . .`
- Ejemplo: undo/redo limitado, queries offline con rollbacks

Persistencia Completa

- Se puede modificar **cualquier versión**
- Cada update desde `vi` **ramifica** creando `vj`
- El timeline es un **árbol** de versiones
- Nuestro segment tree con path copying es **completamente persistente**



Persistencia Confluente

- Además de modificar cualquier versión, se pueden **combinar** dos versiones en una nueva
- El timeline forma un **DAG** (Directed Acyclic Graph)
- Mucho más compleja: el overhead por operación crece con $e(v)$
- **Problema abierto:** ¿se puede lograr $O(\log n)$ de overhead en general?

Persistencia Funcional

- Las estructuras son **inmutables por diseño**
- Cada "modificación" crea una nueva copia
- Ejemplo: listas enlazadas en Haskell/OCaml
- Ventaja: no requiere técnicas especiales, la persistencia es natural
- El path copying es esencialmente una técnica funcional aplicada a estructuras mutables

Persistencia y Análisis Amortizado

No se llevan bien. ¿Por qué?

El problema con la amortización

El análisis amortizado asume que los **créditos** (o potencial) se gastan **una sola vez**.

Ejemplo — Cola con dos listas (two-list queue):

- `push` : $O(1)$ amortizado
- `pop` : $O(1)$ amortizado... pero $O(n)$ en el peor caso (cuando hay que invertir la lista trasera)

El desastre

Guardas una versión justo antes del pop caro y ejecutas:

```
q1 = pop(q)    // O(n) – invierte la lista
q2 = pop(q)    // O(n) – ¡misma inversión desde q!
q3 = pop(q)    // O(n) – ¡otra vez!
...
qn = pop(q)    // O(n)
```

Costo real: $O(n^2)$. **Costo amortizado prometido:** $O(n)$.

Los créditos ya se gastaron en la primera operación.

Consecuencia

Para estructuras de datos persistentes, se necesitan cotas **worst-case**, no amortizadas.

El **path copying** del segment tree persistente da **$O(\log n)$ worst-case** por operación.

No depende de créditos ni potencial: funciona siempre.

Problema G — Persistent Queue

Enunciado típico: Implementar una cola con las siguientes operaciones, donde cada una **crea una nueva versión**:

- `push(v, x)` → versión `w`: mete `x` al final de la cola en la versión `v`
- `pop(v)` → versión `w`: elimina el frente de la cola en la versión `v`
- `front(v)` → `x`: devuelve el elemento al frente de la cola en la versión `v`

Hasta 5×10^5 operaciones.

Solución con Segment Tree Persistente

Idea: Modelar la cola como un array con dos punteros `[front, back)`.

- Cada posición del array guarda un elemento de la cola
- `push` : escribe en `pos[back]` y avanza `back`
- `pop` : avanza `front` (no borramos, solo ignoramos)
- `front` : lee `pos[front]`

Usamos el **segment tree persistente** como el array de posiciones.

Estructura auxiliar

Para cada versión guardamos los punteros y la raíz del ST:

```
struct Version {  
    int root;        // raíz del segment tree para esta versión  
    int front;       // índice del primer elemento  
    int back;        // índice donde se insertará el próximo elemento  
};  
Version ver[MAX_VERSIONS];
```

El segment tree opera sobre el rango `[0, MAX_OPS]` (máximo de pushes posibles).

Push(v, x) → nueva versión w

```
int push(int v, int x) {  
    Version cur = ver[v];  
    // Escribir x en la posición cur.back  
    int new_root = update(cur.root, 0, MAX_OPS, cur.back, x);  
    ver[++vcnt] = {new_root, cur.front, cur.back + 1};  
    return vcnt;  
}
```

Pop(v) → nueva versión w

```
int pop(int v) {  
    Version cur = ver[v];  
    // "Eliminar" el frente: solo avanzamos el puntero  
    ver[++vcnt] = {cur.root, cur.front + 1, cur.back};  
    return vcnt;  
}
```

No tocamos el segment tree. El dato sigue ahí, pero queda fuera del rango [front, back) .

Front(v) → elemento al frente

```
int front(int v) {  
    Version cur = ver[v];  
    // Leer la posición cur.front (es una hoja del segment tree)  
    return query(cur.root, 0, MAX_OPS, cur.front, cur.front);  
}
```

Query puntual en $O(\log n)$. Alternativamente, podemos guardar el valor en `Version` al hacer push y consultarlo en $O(1)$.

Complejidad final

Operación	Tiempo	Nodos nuevos
push	$O(\log N)$	$O(\log N)$
pop	$O(1)$	0
front	$O(1)^*$	0

* $O(1)$ si guardamos el valor en la metadata de la versión; $O(\log N)$ si consultamos al segment tree.

Espacio total: $O(N \log N)$ donde N = cantidad de operaciones.

Resumen

- **Segment tree normal:** $O(\log n)$ por operación, array implícito, sin historia
- **Segment tree persistente:** path copying, $O(\log n)$ worst-case, $O(n + Q \log n)$ memoria
- **Tipos de persistencia:** parcial $\rightarrow$ completa $\rightarrow$ confluente $\rightarrow$ funcional
- **Amortización + persistencia = $\times$** — usar cotas worst-case
- **Persistent Queue:** modelar con ST persistente + punteros front/back

Referencias

- USACO Guide — Persistent Data Structures
- Neelmishra — Persistent Segment Tree
- MIT 6.851 — Advanced Data Structures (Lecture 1)
- Cornell CS 3110 — Amortized Analysis & Persistence (§9.3.4)
- youkn0wwho.academy — Persistent Segment Tree Topic List

¿Preguntas?

Gracias por su atención